DNP3 SEL-751 Experiment — Code & Files Walkthrough

Every script and file explained, block by block, with examples

Companion to PROJECT_HOLISTIC_REPORT — branch research/caseA-ditto-queue

2026-07-23

This companion explains the **code and the files** that produced the results in the main report. It assumes no prior knowledge of the libraries. Read it with the main report open: that one tells you *what we found*; this one tells you *how the software did it*, line by relevant line, with worked examples. Everything here lives under `research/physical_sel751/`.

0. How the pieces fit together

There are three kinds of software here, and they run in a chain:

1. **Collectors** run on the lab machines and touch the real relay: a *probe* (a DNP3 master written with the `pydnp3` library) that sends the reads, and `tcpdump` that records the wire. They produce **raw evidence** — a packet capture (`.pcap`) and an application-side log (`.jsonl`).
2. **The analyzer** (`analyze_clrt.py`) runs offline on the dev box. It reads the raw evidence, reconstructs each request/response transaction, computes timing and statistics, and writes **derived tables and plots** (`per_poll.csv`, `summary.*`, `plots/`).
3. **The validators** (`validation/*.py`) run offline on the *committed* derived tables and the original historical capture. They never touch the relay. They produce the second-opinion analyses (autocorrelation, bootstrap, historical reconciliation).

Why two raw files instead of one? The application library (`pydnp3`) can tell us *what* the relay said (how many data points, what status flags) but not *precisely when* each byte crossed the wire. `tcpdump` gives exact wire timestamps but does not decode DNP3. So we capture both and **merge them by transaction order** in the analyzer: the Nth request in the packet capture is the Nth poll in the application log. The wire file is the authority on *timing*; the application file is the authority on *decoded content*.

1. The directory map

```
research/physical_sel751/
  native_class0_probe.py            single safe poll (the first live contact)
  SEL751_DIRECT_CONNECTIVITY_REPORT.md   the connectivity saga + baseline
  SEL751_CONNECTION_TOPOLOGY.html   the wiring figure
  evidence/                         raw evidence from the single-poll work
    native_class0_v2.pcap             the first clean transaction (wire)
    native_class0_v2_soe.csv          the decoded 69 points
```

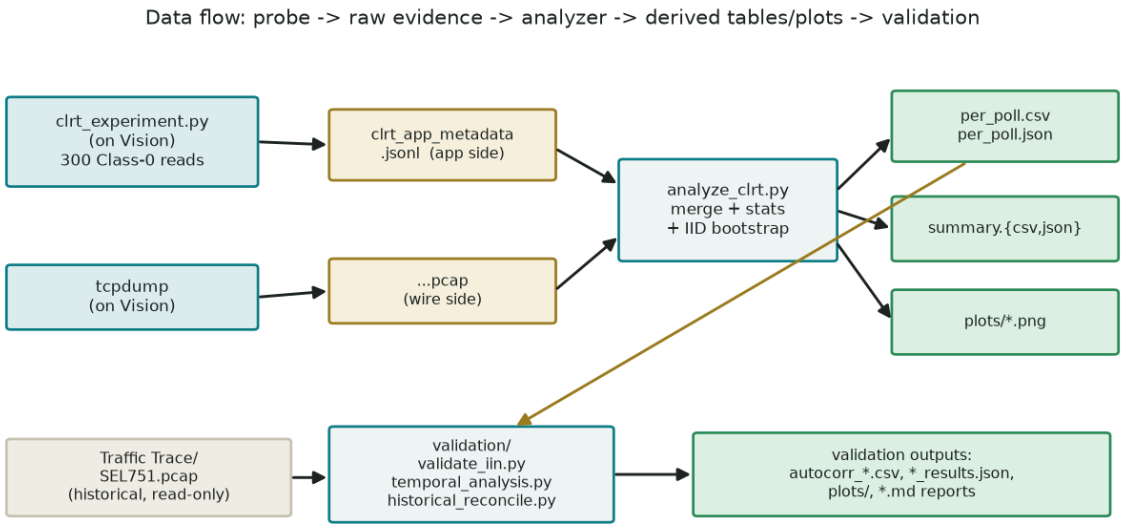

Figure 1: Data flow. The probe and tcpdump produce two raw files (gold); the analyzer merges them into derived tables and plots (green); the validators re-read the derived tables and the historical trace to produce the second-opinion reports.

```
  native_probe_v2.out                the probe's console log
  native_v2_SHA256SUMS.txt           integrity manifest
clrt_300poll_20260723T152242/     the 300-poll experiment (timestamped)
  clrt_experiment.py                 the 300-poll probe (collector)
  analyze_clrt.py                    the analyzer
  per_poll.csv / per_poll.json       one row per poll (derived)
  summary.csv / summary.json         aggregate statistics (derived)
  SHA256SUMS.txt                     integrity manifest for the whole run
  CLRT_EXPERIMENT_REPORT.md          the run's written report
  evidence/                          raw: the .pcap, the .jsonl, logs
  plots/                             histogram, ECDF, box/violin, time-series
  diagrams/                          report diagrams + make_diagrams.py
  validation/                        the validation pass
    validate_iin.py                    decode the status bytes
    temporal_analysis.py               autocorrelation + bootstrap
    historical_reconcile.py            re-analyze the old ~13 ms trace
    autocorr_*.csv, *_results.json     their outputs
    plots/                             ACF, rolling, trend plots
    *_VALIDATION.md / *_RECONCILIATION.md   the written reports
PROJECT_HOLISTIC_REPORT.{md,pdf,html}  the main report
CODE_AND_FILES_WALKTHROUGH.md          this document
```

The **timestamped** directory name (`clrt_300poll_20260723T152242`) is deliberate: every experiment run gets its own dated folder, so a re-run never overwrites old evidence.

2. A five-minute primer on the libraries

You need just enough of four tools to read the code.

- **`pydnp3` / `opendnp3`** — a C++ DNP3 stack with Python bindings. You build a *manager*, add a *channel* (a TCP connection), add a *master* (the DNP3 logic), then call methods like `ScanClasses(...)` to send a read. It runs on its own background thread and calls *your* callback objects when things happen (a response arrives, a task finishes). This "it calls you back on another thread" model is why the experiment code uses a `threading.Event` to hand data safely back to the main thread.
- **`scapy`** — a Python packet library. `PcapReader(file)` yields packets; `p[IP]`, `p[TCP]` access header fields; `bytes(p[TCP].payload)` gives the raw application bytes. We parse DNP3 out of those raw bytes by hand.
- **`tshark`** — Wireshark's command line. We shell out to it only for its expert TCP analysis (retransmissions, duplicate ACKs) which it does better than hand-rolled code.
- **`numpy` / `scipy`** — arrays and statistics (percentiles, the chi-square distribution for Ljung-Box, linear regression). `matplotlib` draws the plots.

3. `native_class0_probe.py` — the safe single poll

This is the first script that ever talked to the physical relay. Its whole job is to send **one** Class-0 read and record the answer, while guaranteeing it can never write to or disturb the relay. It is 85 lines. The interesting parts:

The connection, with a no-retry transport.

```
manager = asiodnp3.DNP3Manager(1, asiodnp3.ConsoleLogger().Create())
# NO-RETRY: initial connect happens immediately; on any close the next attempt is 1 hour
 ↪  out,
# so a relay-side drop cannot trigger reconnection within the capture window (one TCP
 ↪  session).
retry = asiopal.ChannelRetry(openpal.TimeDuration().Seconds(3600),
                             openpal.TimeDuration().Seconds(3600))
channel = manager.AddTCPClient("tcpclient", FILTERS, retry, HOST, LOCAL, PORT, listener)
```

`ChannelRetry(min, max)` sets how long the library waits before reconnecting after a drop. The library *always* reconnects; the trick is to set both bounds to 3600 seconds so, in practice, it reconnects *once* and never again during our few-second window. This is the direct fix for the "434-session storm" described in the main report: with the default retry, a relay that closes instantly makes the library reconnect ~55 times a second.

The safety pins — turning off every automatic behaviour. This block is the heart of "read only":

```
stack = asiodnp3.MasterStackConfig()
stack.master.responseTimeout = openpal.TimeDuration().Seconds(RESP_TIMEOUT_SEC)
stack.master.startupIntegrityClassMask = opendnp3.ClassField()    # no startup poll
stack.master.unsolClassMask = opendnp3.ClassField()               # no ENABLE_UNSOLICITED
 ↪  (0x14)
stack.master.disableUnsolOnStartup = False                        # no DISABLE_UNSOLICITED
stack.master.ignoreRestartIIN = True                              # no WRITE to clear
 ↪  restart IIN
```

```
stack.master.timeSyncMode = getattr(opendnp3.TimeSyncMode, "None")  # no time-sync WRITE
stack.link.LocalAddr = MASTER_ADDR       # = 1
stack.link.RemoteAddr = OUT_ADDR         # = 0
```

Each line disables a default that would otherwise send something to the relay:

- `startupIntegrityClassMask = ClassField()` (empty) — do **not** auto-poll all classes on connect. We want to control exactly what is sent.
- `unsolClassMask = ClassField()` (empty) — do **not** send `ENABLE_UNSOLICITED`. By default the library asks the outstation to start pushing event data; we forbid that.
- `disableUnsolOnStartup = False` — do **not** send `DISABLE_UNSOLICITED` either. The default is `True`, i.e. it *would* send one.
- `ignoreRestartIIN = True` — **the most important pin.** When an outstation reports "I restarted" (a status bit), the library's default is to clear it by sending a **WRITE**. A WRITE to a protection relay is a state change — exactly what a read-only experiment must never do. Setting this `True` means the restart bit is simply left alone (which is why every response in the run shows `DEVICE_RESTART` set — see the IIN section).
- `timeSyncMode = None` — do not send a time-synchronization WRITE.
- `LocalAddr = 1`, `RemoteAddr = 0` — our master's DNP3 link address and the relay's. Note the outstation is **0**, its real configured value, not the `10` from the old captures.

  Worked example — why "read only" is subtle. Imagine you connect a fresh master to a relay that just rebooted. The relay says "DEVICE_RESTART" in its first reply. A helpful DNP3 library "cleans up" by writing a zero to the restart flag. That single automatic WRITE is a control action on live substation equipment. The pin `ignoreRestartIIN = True` is what stops it. Without reading the library's defaults, you would never know this WRITE was about to happen.

The one read. After enabling the master and waiting for the link to settle, the script sends exactly one scan and waits a bounded time:

```
master.Enable()
time.sleep(3)  # let TCP + link layer settle before the one request
master.ScanClasses(opendnp3.ClassField(opendnp3.ClassField.CLASS_0),
                   opendnp3.TaskConfig().Default())
time.sleep(5)  # bounded wait for pure-ACK + response (+ app-confirm iff CON set)
```

`ScanClasses(CLASS_0)` is a one-shot "read all static data" request. The decoded points arrive through a callback (`CSVSOEHandler`, reused from the lab's `run_master.py`) which writes them to a CSV. That is the entire safe-contact procedure.

4. `clrt_experiment.py` — the 300-poll collector

This scales the single poll up to 300, over one persistent session, with strict sequencing and stop conditions. Three ideas make it work.

Idea 1 — a shared `State` object crosses the thread boundary. The DNP3 library calls our callbacks on *its* thread; the main loop runs on the Python thread. They communicate through one object and a `threading.Event`:

```
class State:
    def __init__(self):
```

```
        self.task_done = threading.Event()    # set by the callback, waited on by the loop
        self.task_result = None               # SUCCESS / FAILURE_* (why the task ended)
        self.channel_closed = False           # set if the TCP channel drops
        self.last_iin = None                  # the two status bytes from the last
        ↪ response
        self.poll_points = 0                  # how many data points this response decoded
```

Idea 2 — the callbacks stay tiny. They only record and signal; all real work happens on the main thread (a pattern learned from an earlier bug where heavy work in a DNP3 callback hung the stack):

```
class App(opendnp3.IMasterApplication):
    def OnReceiveIIN(self, iin):
        state.last_iin = (int(iin.LSB), int(iin.MSB))    # capture status bytes
        if iin.HasRequestError():
            state.iin_reqerr = True
    def OnTaskComplete(self, info):
        if info.type == opendnp3.MasterTaskType.USER_TASK:
            state.task_result = info.result             # remember why it ended
            state.task_done.set()                       # wake the main loop
    def OnClose(self):
        state.channel_closed = True

class CountSOE(opendnp3.ISOEHandler):
    def Start(self): state.poll_points = 0              # a new response begins
    def Process(self, info, values):                    # count the decoded points
        vc = rm._VISITOR_CLASS_TYPES.get(type(values))
        if vc is not None:
            v = vc(); values.Foreach(v); state.poll_points += len(v.index_and_value)
```

Idea 3 — the main loop enforces "one request outstanding, then wait one second," and stops on anything unexpected. This is the safety-critical core:

```
for poll in range(1, N_POLLS + 1):
    if state.channel_closed: stop_reason = "channel_closed_before_poll"; break
    state.task_done.clear(); state.task_result = None; state.poll_points = 0
    t_issue = time.time()
    master.ScanClasses(opendnp3.ClassField(opendnp3.ClassField.CLASS_0),
                       opendnp3.TaskConfig().Default())          # send exactly one read
    got = state.task_done.wait(timeout=TASK_WAIT_SEC)            # block until it
↪   completes
    t_done = time.time()
    # ...record the poll to JSONL...
    if not got:                                      stop_reason = "task_wait_timeout";
    ↪ break
    if state.task_result != opendnp3.TaskCompletion.SUCCESS:  stop_reason =
    ↪ "completion_fail"; break
    if state.iin_reqerr:                             stop_reason = "iin_request_error";
    ↪ break
    if state.channel_closed:                         stop_reason = "channel_closed"; break
    completed += 1
    if poll < N_POLLS: time.sleep(INTER_POLL_SLEEP)              # 1 s idle, then next
```

Because the loop only issues the next read *after* `task_done` fires, there is never more than one request in flight. Any of four conditions — a hang, a non-success completion, a protocol error in

the status bits, or a dropped channel — ends the run immediately with a recorded reason and no retry. Each poll's application-side facts (completion status, decoded-point count, the two IIN bytes, timestamps) are written as one JSON line to `clrt_app_metadata.jsonl`.

5. `analyze_clrt.py` — the analyzer

This is where the raw files become numbers. 249 lines; four jobs.

Job 1 — parse a DNP3 frame from raw bytes. DNP3 has a fixed byte layout. The parser reads it by offset:

bytes	field	meaning
0–1	`0x05 0x64`	start-of-frame magic (every DNP3 frame)
2	length	link-layer length; `> 5` means there is application data
3	link control	primary/secondary, link function
4–5	destination	DNP3 link address (little-endian)
6–7	source	DNP3 link address (little-endian)
11	app control	FIR/FIN/CON bits + application sequence number
12	app function	`1` = READ, `129` = RESPONSE
13–14	IIN1, IIN2	(responses only) the two status bytes

```
def dnp3(pl):
    if len(pl) < 13 or pl[:2] != b"\x05\x64":
        return None
    ac = pl[11]                                    # the application control byte
    d = dict(link_len=pl[2], dst=pl[4] | (pl[5] << 8), src=pl[6] | (pl[7] << 8),
             fir=(ac >> 7) & 1, fin=(ac >> 6) & 1, con=(ac >> 5) & 1, seq=ac & 0x0f,
             func=pl[12], func_name=FUNC.get(pl[12], pl[12]))
    if pl[12] == 129 and len(pl) >= 15:
        d["iin_lsb"], d["iin_msb"] = pl[13], pl[14]
    return d
```

Worked example — decode one response frame. A response begins `05 64 73 44 01 00 00 00 ... C1 81 80 00 ....` `0x05 0x64` = magic. `0x73` = 115 = link length (so this frame carries application data). `01 00` = destination address 1 (the master). `00 00` = source 0 (the relay). Further in, `0x81` = 129 = **RESPONSE**, and the next two bytes `80 00` are **IIN1 = 0x80, IIN2 = 0x00** — the DEVICE_RESTART status. Every one of the 300 responses decodes to exactly this shape.

Job 2 — reconstruct transactions, and dodge two traps. The function `build_transactions` walks the packets and pairs each request with its pure ACK and its response. Two subtleties, both learned from bugs:

```
def is_app(pl):  return len(pl) >= 13 and pl[:2] == b"\x05\x64" and pl[2] > 5
def is_req(pl):  return is_app(pl) and pl[12] == 1      # app READ
def is_resp(pl): return is_app(pl) and pl[12] == 129    # app RESPONSE
...
for p in pkts:
    keyd = (p["src"], p["seq"], len(p["pl"]))            # a per-packet identity
    if p["src"] == "VIS" and is_req(p["pl"]):
        if keyd in seen: retrans_req += 1                # a re-sent packet =
        ↪ retransmission
        else: seen.add(keyd); reqs.append(p)
    elif p["src"] == "RELAY" and is_resp(p["pl"]):
        ...
```

- **Trap 1 — link frames also start `0x0564`.** The `pl[2] > 5` test (link length greater than 5) is what separates a real application frame from a pure link-status housekeeping frame. Without it, the session-opening link handshake was mis-counted as a request, shifting every poll by one. This is the "off-by-one" bug from the main report, fixed in one predicate.
- **Trap 2 — retransmissions.** A packet re-sent by TCP has the *same* sequence number. Keying on `(direction, tcp_seq, length)` and skipping duplicates means a retransmission is *counted* but does not create a phantom extra transaction.

Pairing is then done by time: for request i, its response is the first relay application frame after it and before request $i+1$; its pure ACK is the first zero-payload relay ACK in that window. Because the polls are one-at-a-time with a one-second gap, this temporal pairing is unambiguous.

Job 3 — statistics and the (IID) bootstrap. For each timing series it computes mean, median, standard deviation, the percentiles, and a bootstrap confidence interval:

```
def stats_block(x):
    x = np.asarray([v for v in x if v is not None], float)
    def boot(fn, n=10000):
        idx = RNG.integers(0, len(x), size=(n, len(x)))   # 10000 resamples of size
↪   len(x)
        vals = fn(x[idx], axis=1)                          # the statistic on each
↪   resample
        return [float(np.percentile(vals, 2.5)), float(np.percentile(vals, 97.5))]
    return dict(count=len(x), mean=float(np.mean(x)), median=float(np.median(x)),
                std=float(np.std(x, ddof=1)), p90=float(np.percentile(x, 90)),
                bootstrap_ci95_mean=boot(np.mean), bootstrap_ci95_median=boot(np.median),
↪   ...)
```

The line `RNG.integers(0, len(x), size=(n, len(x)))` builds 10,000 rows, each a random resample of the 300 values *with replacement*; `fn(x[idx], axis=1)` computes the statistic on each row; the middle 95% of those is the interval. `RNG` is seeded (`np.random.default_rng(20260723)`) so the numbers are reproducible. **This is the bootstrap the validation pass later flags as anti-conservative** — it assumes independence, which section 7 of the main report disproves.

Job 4 — TCP anomalies via tshark. Rather than re-implement retransmit/duplicate-ACK detection, the analyzer shells out to `tshark` with its expert filters and counts the hits (all zero in our clean run).

6. The validation scripts

`validate_iin.py` (39 lines) — reopens the capture, and for every response frame records the two status bytes as a `(IIN1, IIN2)` pair, then decodes the set bits by name:

```
pairs[(pl[13], pl[14])] += 1        # count each distinct (IIN1, IIN2) pair over all
↪ responses
...
IIN1_bits=[f"IIN1.{k} {NAMES1[k]}" for k in range(8) if i1 & (1 << k)]  # name each set
↪ bit
```

Result: `{(0x80, 0x00): 300}` — one pair, 300 times, one bit (`IIN1.7 DEVICE_RESTART`). It also prints both endian readings (`0x8000` vs `0x0080`) to show why the old notation was ambiguous.

`temporal_analysis.py` (180 lines) — three techniques:

```
def acf(x, maxlag=10):                          # autocorrelation at each lag
    x = np.asarray(x, float); n = len(x); xm = x - x.mean(); denom = np.sum(xm*xm)
    return [float(np.sum(xm[:n-k] * xm[k:]) / denom) for k in range(maxlag + 1)]

def ljung_box(r, n, hs=(5, 10)):                # a formal "is there ANY
↪ autocorrelation?" test
    for h in hs:
        Q = n*(n+2) * sum((r[k]**2)/(n-k) for k in range(1, h+1))
        p = 1 - stats.chi2.cdf(Q, h)            # p-value from the chi-square
↪ distribution
```

- `acf` measures how much a value resembles the value k steps earlier (1.0 = identical pattern, 0 = unrelated). For the CLRT the lag-1 value is 0.35 — clearly not zero.
- `ljung_box` rolls the first h autocorrelations into one number `Q` and asks the chi-square distribution how surprising it is under "no correlation." `p` near 0 means "definitely correlated." (We compute it by hand because `statsmodels` is not installed.)

The **moving-block bootstrap** is the fix for the correlated data. Instead of resampling single polls, it resamples contiguous blocks of length `L`, so each block keeps its internal correlation:

```
def block_boot(x, fn, L, n=10000):
    N = len(x); nb = int(np.ceil(N / L))
    ext = np.concatenate([x, x[:L]])            # wrap around so blocks near the end are
↪ whole
    starts = RNG.integers(0, N, size=(n, nb))   # random block start indices
    samples = ...  # glue nb blocks of length L together, trim to N, compute fn on each
```

Comparing `block_boot` against the ordinary bootstrap is what produced the wider, honest confidence intervals in the main report.

`historical_reconcile.py` (141 lines) — reruns the same transaction walk on the *original* `Traffic Trace/SEL751.pcap`, filtered to the relay's old address `10.0.0.1`, and crucially splits the results by request function code:

```
for f in sorted(set(t["func"] for t in txns)):   # group by READ vs DIRECT_OPERATE
    sub = [t for t in txns if t["func"] == f]
    byfunc[f] = dict(n=len(sub), ...,
        ack_to_response_clrt_ms=stat([t["clrt_ms"] for t in sub if t["clrt_ms"] is not
↪ None]))
```

That split is the decisive test: it showed the historical READ-only CLRT (13.18 ms) matches its control CLRT (12.84 ms), so request type does not explain the 7x gap versus the live 1.9 ms. The script also records IP TTLs and Ethernet MACs — the evidence that led to, and then retracted, the "simulator" idea (both old and new relay use TCP TTL 64).

7. The diagram and report builders

- **`clrt_300poll_.../diagrams/make_diagrams.py`** — draws the report diagrams with matplotlib (small boxes and arrows). Used because the Mermaid command-line renderer needs a headless browser that was unavailable here. Re-run it to regenerate every `diag_*.png`.
- **`PROJECT_HOLISTIC_REPORT_html_builder.py`** — reads the plot PNGs, base64-encodes them into `data:` URIs, and injects them into the HTML report template so the artifact is fully self-contained (no external files).
- **`PROJECT_HOLISTIC_REPORT.md` / `CODE_AND_FILES_WALKTHROUGH.md`** — the Markdown sources that `paper-build.sh` turns into PDFs via pandoc + tectonic (LaTeX).

8. The data and evidence files, with examples

`...pcap` — the raw packet capture (binary). Read it with `tshark -r file.pcap` or scapy. It is the authority on wire timing.

`clrt_app_metadata.jsonl` — one JSON object per line, one per poll (the application side). Example line:

```
{"poll_number": 1, "t_issue": 1784834686.60, "t_done": 1784834686.66,
 "completion": "SUCCESS", "decoded_point_count": 69,
 "iin_lsb": 128, "iin_msb": 0, "iin_request_error": false,
 "channel_closed": false, "error": null}
```

`iin_lsb: 128` is `0x80` = DEVICE_RESTART; `decoded_point_count: 69` is the number of data points in that response; `completion: "SUCCESS"` means the DNP3 task finished cleanly.

`per_poll.csv` — the merged, per-poll table (the main derived artifact). One row per poll with the wire timings and the decoded facts side by side. Key columns: `poll_number`, `request_timestamp`, `pure_tcp_ack_timestamp`, `response_timestamp`, `request_to_ack_ms`, `ack_to_response_clrt_ms`, `request_to_response_ms`, `request_wire_bytes`, `response_wire_bytes`, `dnp3_response_length`, `fir`, `fin`, `con`, `function_code`, `iin_lsb`, `iin_msb`, `decoded_point_count`. This is the file every statistic is computed from, and the file the validators re-read.

`summary.json` / `summary.csv` — the aggregate statistics for the three timing series (mean, median, percentiles, bootstrap intervals) plus the integrity checks (one TCP session, zero retransmissions, all 300 successful).

`autocorr_*.csv` — one file per timing series: the autocorrelation at lags 1–10 with the significance band, and the Ljung-Box result.

`SHA256SUMS.txt` — a cryptographic checksum of every file, so anyone can verify the evidence was not altered. Regenerate/verify with `sha256sum -c SHA256SUMS.txt` (it printed all-`OK` in the validation pass, proving no raw file was touched).

9. How to reproduce, end to end

Every step is a single command. The collectors must run on the lab machines (they touch the relay, gated on authorization); the analysis runs anywhere with the Python research environment.

```
# 1. Collect (on Vision): start tcpdump, then the probe (one persistent session, no
 ↪  retry)
sudo tcpdump -i eno1 -s0 -w /tmp/run.pcap "host 192.168.10.7 and tcp port 20000" &
python3 clrt_experiment.py            # writes /tmp/clrt_app_metadata.jsonl

# 2. Analyze (on the dev box): merge pcap + jsonl -> per_poll.csv, summary, plots
$RESEARCH_PYTHON analyze_clrt.py

# 3. Validate (offline, on the committed evidence)
$RESEARCH_PYTHON validation/validate_iin.py
$RESEARCH_PYTHON validation/temporal_analysis.py
$RESEARCH_PYTHON validation/historical_reconcile.py

# 4. Verify nothing was altered, and rebuild the documents
sha256sum -c SHA256SUMS.txt
python3 diagrams/make_diagrams.py
cd research/physical_sel751 && paper-build.sh PROJECT_HOLISTIC_REPORT.md report.pdf ieee
```

Because the probe pins every automatic behaviour off and uses a no-retry transport, re-running the collector is as safe as the original: one session, read-only, hard-stopping on anything unexpected. Because the bootstrap and block-bootstrap are seeded, the derived numbers reproduce exactly; only the plot PNGs may differ byte-for-byte (matplotlib embeds a timestamp).